76. Minimum Window Substring

Hard · String · Hash Table · Sliding Window

Approach — Sliding Window with Frequency Counting

The problem asks for the smallest substring of s that contains every character of t with the required frequencies. This is a classic **sliding window** problem where a right pointer expands the window to include new characters, and a left pointer shrinks it whenever all required characters are satisfied.

Two frequency arrays `hashT` and `hashS` of size 60 cover both uppercase and lowercase letters (indexed by `c - 'A'`, which spans A–Z at 0–25 and a–z at 32–57). The variable `required` counts how many distinct characters in t need to be matched, and `formed` tracks how many of those are currently satisfied in the window.

Key Insight: A character in t is considered "formed" when its count in the current window equals its count in t — not just present, but sufficiently frequent. This allows the window to contract safely without losing valid matches.

Step-by-step Breakdown

Step 1 — Build the target frequency map

Iterate through t. For each character c, increment `hashT[c-'A']`. Increment `required` only the first time a character appears in t (i.e. when its count transitions from 0 to 1), counting distinct character types.

Step 2 — Expand the window with the right pointer

Advance `r` through s, adding `s[r]` to `hashS`. After adding, if this character is needed by t and its window count now exactly equals its required count, increment `formed`.

Step 3 — Shrink the window with the left pointer

When `formed == required`, the window is valid. Record it if smaller than current minimum using indices `i` and `j`. Then shrink from the left: decrement the count of `s[l]` in `hashS`. If that causes a required character to fall below its needed count, decrement `formed`. Advance `l++` and repeat until the window is no longer valid.

Step 4 — Return the result

After the loop, if `i > j` no valid window was found — return an empty string. Otherwise return `s.substr(i, j-i+1)`, the minimum window recorded.

Solution Code

```
class Solution {
public:
    string minWindow(string s, string t) {
        vector<int> hashT(60), hashS(60);
        int required = 0, formed = 0;
        for(auto& c: t)
        {
            if(!hashT[c-'A']) required++;
            hashT[c-'A']++;
        }
        int l = 0, r = 0, mn = 1e6, i = 1e6, j = -1;
        while(r < s.size())
        {
            hashS[s[r]-'A']++;
            if(hashT[s[r]-'A'] && hashT[s[r]-'A'] == hashS[s[r]-'A']) formed++;
            while(required == formed)
            {
                if(r-l+1 < mn)
                {
                    mn = r-l+1;
                    i = l; j = r;
                }
                hashS[s[l]-'A']--;
                if(hashT[s[l]-'A'] && hashS[s[l]-'A'] < hashT[s[l]-'A']) formed--;
                l++;
            }
            r++;
        }
```

```
29          if(i > j) return "";
30          return s.substr(i, j-i+1);
31      }
32  };
```

Complexity Analysis

TIME COMPLEXITY	SPACE COMPLEXITY
O(|s| + |t|)	**O(1)**
Building the target map is O(|t|). Each character in s is added and removed from the window at most once — left and right pointers each traverse s once. Total: O(|s| + |t|).	The two frequency arrays are fixed size 60, independent of input size. All other variables are constant — O(1) auxiliary space.